Hazard-First Assistive Vision: A Priority-Scheduled, Schema-Verified Perception Pipeline for Visually Impaired Navigation

Author name(s) withheld for double-blind review

Affiliation and contact withheld — ICIICTE-2026 Track 3 submission

***Abstract*—Independent indoor movement confronts a visually impaired person with information that arrives silently: a descending staircase, an obstacle at walking height, a sign that is rotated or perspective-distorted. Systems that only read text or only detect objects address fragments of this problem, and most speak their findings in arrival order rather than in order of danger. We present an assistive vision system, implemented as a smartphone web client and a Python inference server, that integrates oriented scene-text reading, obstacle detection over a navigation-specific class schema, monocular distance estimation expressed in walking steps, and a priority scheduler that speaks hazards before requested objects, signs, and scene summaries. Two design decisions distinguish the system. First, safety by construction: the runtime identifies detector weights by their class names and refuses unrecognized vocabularies, and the dataset tooling refuses a split that declares a class it cannot populate. Second, honest capability reporting: the health endpoint discloses which hazard classes the deployed weights can and cannot raise. A preliminary detector raises frame-level hazard recall from 0.252 to 0.463 over a COCO baseline while improving precision from 0.860 to 0.958—yet is withheld from deployment because person recall regresses at the deployed operating point. Final model training is in progress; the pipeline itself is validated by 152 automated tests.**

***Index Terms*—assistive technology, visually impaired navigation, object detection, scene text recognition, hazard prioritization, monocular distance estimation, system safety verification.**

I. Introduction

The World Health Organization estimates that more than two billion people live with a vision impairment, a substantial fraction of whom cannot resolve the visual cues that sighted pedestrians use continuously: door frames, staircase edges, overhead signage, and the position of people moving nearby [1]. Indoors, where satellite positioning is unavailable and tactile paving is rare, the white cane covers roughly a one-metre radius at floor level and nothing above it, and it reads no text at all.

Computer vision can, in principle, supply the missing information. Modern object detectors localize dozens of object categories in real time [8], [9], and scene-text systems detect and recognize signage under rotation and perspective distortion [2]–[5]. In practice, however, assistive deployments of these components tend to inherit two structural weaknesses. The first is fragmentation: reading applications read, detection applications detect, and neither orders its output by consequence, so the user hears about a menu board and an approaching staircase with equal urgency. The second is silent capability loss: when a detector is swapped or retrained, nothing in a typical serving stack verifies that the new weights can still raise every warning the system's logic assumes—a failure mode that is invisible precisely to the user it endangers.

This paper describes an end-to-end system built around those two observations. A smartphone browser captures frames and voice commands; a server runs detection, oriented text reading, and spatial reasoning; a priority engine composes a single spoken sentence in which danger always precedes convenience. The class schema is navigation-specific—it names staircases, doors, and dustbins that COCO-trained detectors structurally cannot report—and the runtime treats the schema as a contract to be verified, not a filename to be trusted.

Our contributions are: **(1)** a priority-scheduled perception-to-speech pipeline in which hazards interrupt, a critical class (descending stairs) preempts everything, and distant hazard-class objects are demoted so that only proximate danger interrupts the user; **(2)** step-metric monocular distance estimation that fuses two independent estimators—a class-height pinhole model and a ground-plane model with device-pitch correction—by taking the conservative minimum, reported in walking steps, with a two-minute per-device field-of-view calibration; **(3)** a safety-by-construction mechanism spanning runtime and tooling: name-based schema verification that refuses unrecognized weights, split construction that refuses unlearnable classes, and a health endpoint that reports which alerts the deployed model can actually raise; **(4)** a reproducible, leakage-safe dataset pipeline for the navigation schema, including a human-verified re-tagging protocol that gave the safety-critical descending-stairs class its first labelled data. All pipeline logic is pinned by 152 automated tests that run in continuous integration.

II. Related Work

Assistive vision systems. Surveys of assistive technology for blind and low-vision users trace a progression from sonar canes to camera-based scene description [6], [7]. Deployed reading assistants demonstrate demand, but published descriptions emphasize recognition accuracy over information ordering: output is typically spoken in detection order, and obstacle awareness, when present, is not integrated with text reading in a single prioritized channel. Navigation-focused prototypes conversely emphasize obstacle avoidance and path planning [7] but rarely read signage, although signage is how buildings communicate routes to everyone else.

Scene-text detection and recognition. EAST [2] and CRAFT [3] established efficient oriented and character-affinity text detection; DBNet [4] and MOST [5] refined arbitrary-shape detection. Recognition commonly follows the CRNN design [10]. These systems target benchmark protocols, not assistive delivery: they stop at bounding quadrilaterals and transcripts. Our pipeline consumes their output—each detected quadrilateral is perspective-rectified to horizontal before recognition—and treats reading as one prioritized voice among several, not as the product.

Object detection for navigation classes. COCO [8] supplies person, furniture, and vehicle categories with massive supervision, but contains no staircase, door-as-obstacle, or dustbin class; Open Images [11] contains a single undirected Stairs class. Since a descending staircase is the highest-consequence indoor hazard and an ascending one is merely an obstacle, directionality is not a labelling nicety but the difference between "caution" and "stop." We are not aware of prior assistive work that both trains a directional stairs class and verifies, at runtime, that the deployed weights still carry it—the gap this paper addresses.

TABLE I. POSITIONING RELATIVE TO REPRESENTATIVE APPROACHES

Approach	Text	Obstacles	Priority	Verification
Reading assistants [6]	yes	no	no	no
Navigation prototypes [7]	no	yes	partial	no
Text detection [2]–[5]	yes	—	—	—
This work	oriented	nav schema	yes	yes

III. PROPOSED METHODOLOGY

A. System Architecture

The system is client–server (Fig. 1). The client is a browser application on an ordinary smartphone: it captures JPEG frames (continuous scan every ≈2.6 s in live-assist mode, or single-shot), performs speech recognition for commands, renders text-to-speech and vibration, and reads the device gyroscope and GPS. The server, a Python/FastAPI process, exposes one analysis endpoint: a frame plus an optional keyword returns detected objects, hazards, texts, symbols, spatial attributes, and a single composed speech string. Outdoor turn-by-turn navigation is delegated to a maps application by design; the contribution of this work is indoor perception, and re-implementing routing would add no information a maps client does not already speak.

Fig. 1. Overall architecture: smartphone client (capture, voice input, speech and vibration output) and inference server (detection, oriented text reading, spatial reasoning, priority engine). [Insert diagram before submission.]

The server defends its own availability: frames above 8 MB are refused, per-client request rate is limited, and a health endpoint reports model warmup, the active class schema, and—as discussed in Section III-D—which alert paths the loaded weights can actually serve. A failed model load reports unhealthy rather than degrading silently, so an orchestrator restarts a worker that could never answer.

B. Visual Perception

Obstacle detection. A YOLOv8 detector [9] runs over the full frame. The serving layer is schema-agnostic: if fine-tuned weights are present, their class vocabulary is read from the weights themselves; otherwise the stock COCO model serves as a fallback with COCO hazard semantics. The trained schema (AV-7) names seven classes chosen for navigation consequence: person, chair, table, door, stairs_up, dustbin, and stairs_down. Hazard status is a semantic role assigned in code to a class name—deliberately not a visual class, since a generic "obstacle" category has no consistent appearance for annotators to agree on.

Oriented text reading. Text detection currently uses the CRAFT detector via EasyOCR [12], which yields rotated quadrilaterals natively; each quadrilateral is perspective-warped to horizontal before recognition by the stock CRNN recognizer [10]. No recognizer fine-tuning is claimed. The pipeline is detector-agnostic: a YOLOv8-OBB text detector can replace CRAFT by placing weights at a designated path, and conversion tooling for ICDAR-2015 [13] is implemented and tested; training that detector is in progress and is reported here as pipeline design, not as a trained result.

Symbol resolution. Signs resolve through two routes: a detected class implies its symbol, and recognized text is matched against a symbol vocabulary by whole-word comparison. The whole-word rule exists because substring matching once announced a restaurant MENU board as a washroom—"men" is a substring of both MENU and WOMEN—and the regression is pinned by a test. Five pictogram sign classes exist in the annotation vocabulary but hold no labelled boxes yet, so no trained symbol recognition is claimed; text-free pictograms are the stated next annotation target precisely because they are the case OCR-only readers cannot serve.

C. Spatial Awareness in Walking Steps

Direction is derived from frame thirds (left / ahead / right). Distance uses two independent monocular estimators: (i) a pinhole model over per-class real-world height priors [14], and (ii) a ground-plane model that projects the bounding-box foot point through the camera's vertical field of view, corrected by the device pitch reported by the gyroscope. The two estimates are fused by taking the minimum: under-estimating distance is the safe error for a person who cannot visually confirm the estimate. The result is spoken in walking steps—the unit the user can act on—rather than metres. Because browser-reported field of view is unreliable across devices, the client includes a one-tap calibration: the user places any known object four steps ahead, and the true vertical field of view is solved from that single measurement and persisted per device.

D. Safety-Priority Engine and Verified Capability

The priority engine composes one sentence per frame in a fixed order: critical alert → hazards → user-requested object → symbols and signs → scene summary. The single critical class,

stairs_down, produces an interrupting "Warning—stop and proceed carefully" utterance that preempts all other content. Hazard-class objects that are distant are demoted to ordinary objects so that only proximate danger interrupts; the client additionally vibrates on hazards and suppresses its own microphone while speaking, so the recognizer does not transcribe the system's own output.

Two verification mechanisms make this pipeline trustworthy across model updates. First, the runtime identifies any weights placed at the custom-model path by their class names, never the filename. Weights whose vocabulary matches the navigation schema activate navigation-hazard semantics; COCO weights retain COCO hazard semantics; anything else is refused at startup. The bug this kills is concrete: a COCO model mistaken for the fine-tune would silently stop flagging vehicles, with no error raised to a user who cannot see the difference. Second, the same philosophy is applied before training: the split builder refuses a dataset that declares a class with no training boxes, because the resulting model would pass the name check while structurally unable to emit the class it claims. Between the two checks, the health endpoint reports the active schema, whether the critical-alert path is currently servable, and the list of hazard roles the deployed weights cannot raise—capability loss is surfaced as data, not discovered by accident.

IV. Implementation and Dataset

Software. Python server (FastAPI, PyTorch, Ultralytics YOLOv8, EasyOCR); browser client in plain HTML/JavaScript using the Web Speech, vibration, device-orientation, and geolocation APIs. The repository ships a container image verified end-to-end (it builds, serves, and answers real frames), and a continuous-integration workflow runs the full test suite on an independent platform (Ubuntu, Python 3.11) from the development machine.

Corpus. The self-collected corpus comprises 1,202 frames extracted at 0.5 fps from eight indoor walkthrough videos (malls, hospitals, campuses: two public tours, six self-recorded), blur- and duplicate-filtered, and pre-labelled by a stock YOLOv8s so that annotation is correction rather than drawing; 219 ambiguous pre-label boxes were individually adjudicated in a purpose-built review tool. This is supplemented by 1,451 Open Images V7 [11] images supplying volume for door, staircase, and dustbin.

A human-verified critical class. Open Images labels staircases without direction. All 579 imported staircase boxes were re-tagged for this work under a two-stage protocol: a vision-model pass proposed a direction for every box, and a human then reviewed every box proposed as descending, confirming 65, demoting 53, and dropping one—the audit record ships with the dataset. The asymmetric protocol reflects asymmetric risk: a wrongly ascending label leaves the status quo, while a wrongly descending label would train the class that triggers the system's only interrupting alert. The final corpus therefore carries 65 verified descending-stairs boxes (60 train / 5 validation)—few, but the first supervised signal this class has had, and honestly below the support at which we treat per-class AP as more than directional.

Split. Train/validation splitting is by whole video, never by frame: consecutive 0.5 fps frames are near-duplicates, and a frame-level split leaks validation into training. Open Images contributes its own upstream train/validation assignment, which is honoured rather than re-split. The current split holds 1,685 training and 336 validation frames; four self-recorded videos are held out entirely for validation. Per-class support (train/val boxes): person 2,533/266; door 604/288; dustbin 730/30; stairs_up 399/27; chair 277/28; table 75/17; stairs_down 60/5. Seven further names (five pictogram signs, pole, signboard) remain annotation-vocabulary only; signboard was retired from the trained schema after 50 boxes measured AP@50 of 0.000—a declared-but-unlearnable class costs macro-mAP while the name-based check still certifies the weights.

V. Evaluation and Results

A. Functional Validation

All deterministic pipeline logic is pinned by 152 automated tests (passing locally and in CI): priority ordering and the critical interrupt; hazard demotion by proximity; step-distance estimation including a portrait-mode regression; deskew geometry; symbol whole-word matching; API limits (oversize frames, rate limiting, warmup refusal); schema verification for matching, COCO, and unknown vocabularies; and every dataset tool (remapping, adjudication merge, by-name reindexing, leakage-safe splitting, coverage gating). End-to-end behaviour was verified inside the shipped container: health reporting, real-frame analysis with detection, OCR, step distance, and keyword search, and rejection of oversize uploads.

B. Model Training and Quantitative Evaluation

Model development is staged, and this paper reports the stage each artefact has actually reached. A preliminary detector (YOLOv8n, 640 px, early-stopped at epoch 36) exists and is fully measured; the final detector (YOLOv8s, 832 px, 120 epochs, GPU) and a preliminary model for the extended seven-class schema are training at the time of writing.

TABLE II. Preliminary Detector vs. COCO Baseline (Held-Out Split, Laptop CPU)

Metric	COCO baseline	Prelim. fine-tune
Hazard-frame precision	0.860	0.958
Hazard-frame recall	0.252	0.463
Hazard F1	0.389	0.624
False-alarm frames	6	3
mAP@50 (6 classes)	n/a (vocab differs)	0.334
Detection latency p50	321 ms	102 ms
End-to-end latency p50	4,262 ms	3,330 ms

Per-class AP@50 with validation support: dustbin 0.694 (30); stairs_up 0.471 (46); door 0.384 (288); chair 0.277 (28); person 0.167 (266); table 0.011 (17); classes under 30 validation boxes are flagged as noise by our harness. The classes COCO structurally cannot report—dustbin, staircase, door—move from 0.000 deployed recall to 1.000, 0.269, and 0.210 respectively.

A negative result, reported rather than buried. In the same run, person recall at the deployed operating point falls from 0.164 to 0.061: the fine-tuning corpus holds 2,533 indoor person boxes against COCO's millions. The aggregate hazard metric improves regardless, because newly detected classes compensate—which is exactly why reporting only the aggregate would misrepresent a system materially worse at detecting people. The preliminary model is therefore withheld from deployment, and subsequent analysis indicates the deployed per-class confidence threshold (calibrated for COCO weights) accounts for part of the regression; threshold recalibration is being evaluated alongside the final training run.

TABLE III. FINAL-MODEL METRICS: STATUS

Metric	Status
Precision / Recall / F1 (final model)	pending final validation
mAP@50, mAP@50–95 (final model)	pending final validation
Per-class AP incl. stairs_down	pending final validation
GPU / hosted-tier latency	pending measurement

All pending values are produced by the repository's own evaluation harness on the same held-out split as Table II; none will be sourced from literature.

C. Discussion

The measured pipeline latency (p50 ≈ 3.3 s per frame on a laptop CPU) is dominated by text recognition (≈3.2 s); detection contributes ≈0.1 s. The system is honestly a paced assistant, not a real-time one, and the latency target for future work is the OCR stage, not the detector. Dataset limits are equally plain: single-region footage, sparse support for table, chair, and above all stairs_down (5 validation boxes—its AP is a direction, not a claim), and no trained pictogram-sign recognition yet. Distance estimation is monocular and assumes the object rests on the ground plane; the conservative minimum-fusion policy trades accuracy for safety by design. No user study has been conducted, and none is claimed; a protocol (task completion, collisions, usability scoring [16] against a priority-disabled baseline) is specified for post-training work.

VI. CONCLUSION

We presented an assistive vision system that treats what to say first and whether the model can still say it as first-class engineering problems. The implemented pipeline integrates oriented text reading, navigation-specific obstacle detection, step-metric monocular distance, and prioritized speech, and wraps the model lifecycle in verification: schemas are checked by name at startup, unlearnable classes are refused before training, and the health endpoint reports precisely which alerts the deployed weights can raise. A preliminary detector nearly doubles frame-level hazard recall over a COCO baseline at higher precision, and its person-recall regression—detected by the same evaluation discipline—is the documented reason it is not deployed. The safety-critical descending-stairs class now carries its first human-verified labels through an asymmetric verification protocol. Final model training is in progress; its metrics will complete Tables II–III without altering the system's architecture or claims.

REFERENCES

[1] World Health Organization, World Report on Vision. Geneva, Switzerland: WHO, 2019.

[2] X. Zhou et al., "EAST: An efficient and accurate scene text detector," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2017, pp. 5551–5560.

[3] Y. Baek, B. Lee, D. Han, S. Yun, and H. Lee, "Character region awareness for text detection," in Proc. IEEE CVPR, 2019, pp. 9365–9374.

[4] M. Liao, Z. Wan, C. Yao, K. Chen, and X. Bai, "Real-time scene text detection with differentiable binarization," in Proc. AAAI Conf. Artif. Intell., 2020, pp. 11474–11481.

[5] M. He et al., "MOST: A multi-oriented scene text detector with localization refinement," in Proc. IEEE CVPR, 2021, pp. 8813–8822.

[6] A. Bhowmick and S. M. Hazarika, "An insight into assistive technology for the visually impaired and blind people: State-of-the-art and future trends," J. Multimodal User Interfaces, vol. 11, no. 2, pp. 149–172, 2017.

[7] S. Real and A. Araujo, "Navigation systems for the blind and visually impaired: Past work, challenges, and open problems," Sensors, vol. 19, no. 15, p. 3404, 2019.

[8] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," in Proc. Eur. Conf. Comput. Vis. (ECCV), 2014, pp. 740–755.

[9] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[10] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition," IEEE Trans. Pattern Anal. Mach. Intell., vol. 39, no. 11, pp. 2298–2304, 2017.

[11] A. Kuznetsova et al., "The Open Images Dataset V4: Unified image classification, object detection, and visual relationship detection at scale," Int. J. Comput. Vis., vol. 128, pp. 1956–1981, 2020.

[12] JaidedAI, "EasyOCR: Ready-to-use OCR," 2020. [Online]. Available: https://github.com/JaidedAI/EasyOCR

[13] D. Karatzas et al., "ICDAR 2015 competition on robust reading," in Proc. Int. Conf. Document Anal. Recognit. (ICDAR), 2015, pp. 1156–1160.

[14] R. Hartley and A. Zisserman, Multiple View Geometry in Computer Vision, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2004.

[15] A. Gupta, A. Vedaldi, and A. Zisserman, "Synthetic data for text localisation in natural images," in Proc. IEEE CVPR, 2016, pp. 2315–2324.

[16] J. Brooke, "SUS: A 'quick and dirty' usability scale," in Usability Evaluation in Industry. London, U.K.: Taylor & Francis, 1996, pp. 189–194.